



TRƯỜNG ĐẠI HỌC THƯƠNG MẠI

Khoa HTTT Kinh tế và THMĐT

Bộ môn Công nghệ thông tin

Chương 3

THIẾT KẾ CA KIỂM THỬ



Nội dung

1. Kiểm thử hộp đen (kiểm thử chức năng)
 - Kiểm thử lớp tương đương
 - Kiểm thử giá trị biên
 - Kiểm thử bảng quyết định
 - Kiểm thử dựa trên đặc tả use cases
2. Kiểm thử hộp trắng (kiểm thử cấu trúc)
 - Kiểm thử luồng điều khiển
 - Kiểm thử luồng dữ liệu



PHẦN II: KIỂM THỬ HỘP TRẮNG



Kiểm thử hộp trắng

- Đối tượng được kiểm thử là 1 hàm, 1 modun, 1 phân hệ chức năng
- Dựa vào giải thuật cụ thể, cấu trúc dữ liệu bên trong của đối tượng được kiểm thử để xác định đối tượng đó có thực hiện đúng không
- Người kiểm thử cần hiểu rõ về giải thuật, ngôn ngữ lập trình, cấu trúc dữ liệu bên trong để hiểu chi tiết về đoạn code cần kiểm thử



Kiểm thử hộp trắng

- Thường được dùng để kiểm thử mức đơn vị
 - Lập trình hướng đối tượng: kiểm thử từng method của class
- Tốn quá nhiều thời gian, công sức nếu áp dụng cho kiểm thử tích hợp, kiểm thử chức năng
- Hai kỹ thuật chính:
 - Kiểm thử luồng điều khiển: kiểm thử giải thuật
 - Kiểm thử luồng dữ liệu: kiểm thử đời sống của từng biến



Kiểm thử hộp trắng

- Kiểm thử hộp trắng dựa trên mã nguồn để xây dựng các ca kiểm thử và kiểm tra đầu ra.
- Các khái niệm liên quan:
 - Đồ thị
 - Đường thi hành (execution paths)
 - Độ bao phủ (coverage)
 - Luồng điều khiển
 - Luồng dữ liệu



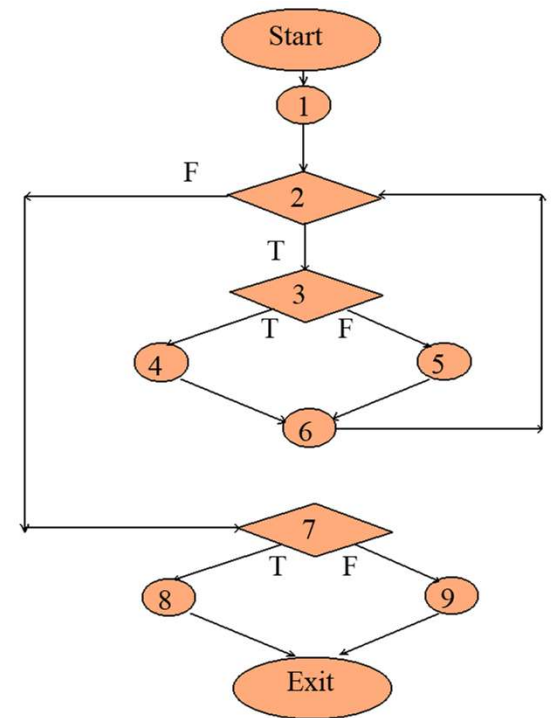
4. Ưu điểm và nhược điểm của kiểm thử hộp trắng

- Ưu điểm
 - Test có thể bắt đầu ở giai đoạn sớm hơn, không cần phải chờ đợi cho GUI để có thể test
 - Test kỹ càng hơn, có thể bao phủ hầu hết các đường dẫn
 - Thích hợp trong việc tìm kiếm lỗi và các vấn đề trong mã lệnh
 - Cho phép tìm kiếm các lỗi ẩn bên trong
 - Các lập trình viên có thể tự kiểm tra
 - Giúp tối ưu việc mã hoá
 - Do yêu cầu kiến thức cấu trúc bên trong của phần mềm, nên việc kiểm soát lỗi tối đa nhất.
- Nhược điểm
 - Vì các bài kiểm tra rất phức tạp, đòi hỏi phải có các nguồn lực có tay nghề cao, với kiến thức sâu rộng về lập trình và thực hiện.
 - Maintenance test script có thể là một gánh nặng nếu thể hiện thay đổi quá thường xuyên.
 - Vì phương pháp thử nghiệm này liên quan chặt chẽ với ứng dụng đang được test, nên các công cụ để phục vụ cho mọi loại triển khai / nền tảng có thể không sẵn có.

Định nghĩa đồ thị chương trình

“Cho một chương trình trong ngôn ngữ mệnh lệnh, đồ thị chương trình của nó là một đồ thị có nhãn, có hướng trong đó

- **Đỉnh** là các nhóm lệnh hoặc một phần của một lệnh
- **Cạnh** là luồng điều khiển”
- Nếu i và j là hai đỉnh thì có một cạnh từ i đến j trong đồ thị chương trình nếu và chỉ nếu lệnh ở đỉnh j có thể chạy ngay sau (các) lệnh ở đỉnh i .



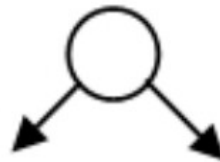
Các thành phần của đồ thị



Điểm xuất phát



Khối xử lý



Điểm quyết định



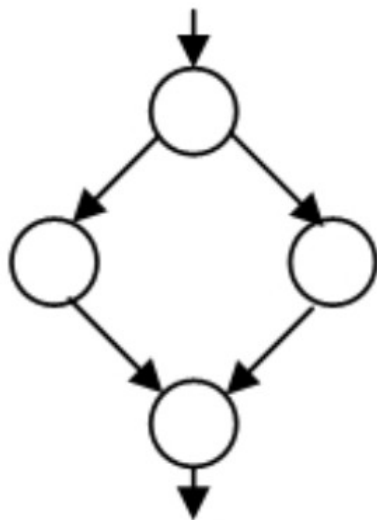
Điểm nối



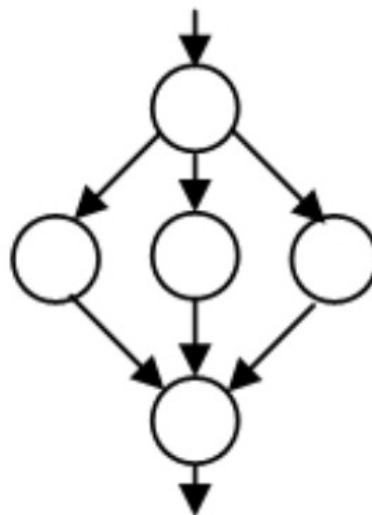
Điểm kết thúc



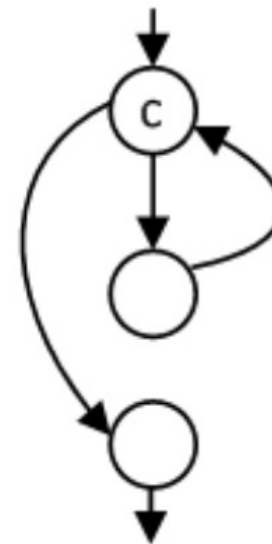
Tuần tự



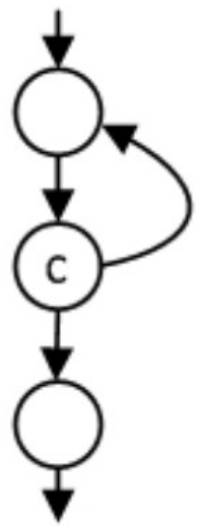
Rẽ nhánh



switch



while c do ...

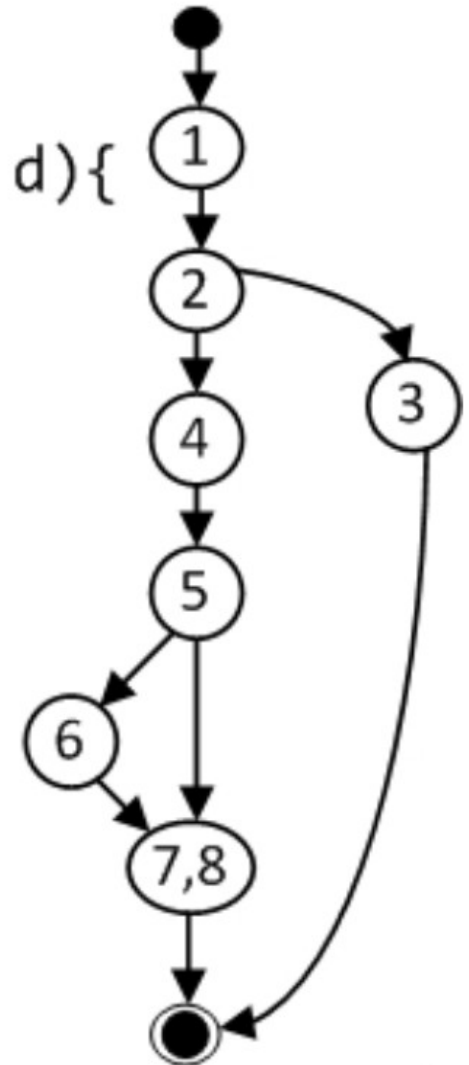


do ... while c



Ví dụ

```
float foo(int a, int b, int c, int d){  
1.  float e;  
2.  if (a==0)  
3.      return 0;  
4.  int x = 0;  
5.  if ((a==b) || (c==d))  
6.      x = 1;  
7.  e = 1/x;  
8.  return e;  
}
```





Đồ thị chương trình

- Nhóm các lệnh tạo thành một đỉnh trong đồ thị chương trình gọi là khối cơ bản.
- Dễ dàng xây dựng các khối cơ bản và tạo đồ thị chương trình



Ví dụ 3 các khối cơ bản

```
FindMean (FILE ScoreFile)
```

```
{ float SumOfScores = 0.0;
```

```
int NumberOfScores = 0;
```

```
float Mean=0.0; float Score;
```

```
Read(ScoreFile, Score);
```

```
while (! EOF(ScoreFile) {
```

```
if (Score > 0.0 ) {
```

```
SumOfScores = SumOfScores + Score;
```

```
NumberOfScores++;
```

```
}
```

```
Read(ScoreFile, Score);
```

```
}
```

```
/* Compute the mean and print the result */
```

```
if (NumberOfScores > 0) {
```

```
Mean = SumOfScores / NumberOfScores;
```

```
printf(" The mean score is %f\n", Mean);
```

```
} else
```

```
printf ("No scores found in file\n");
```

```
}
```

1

4

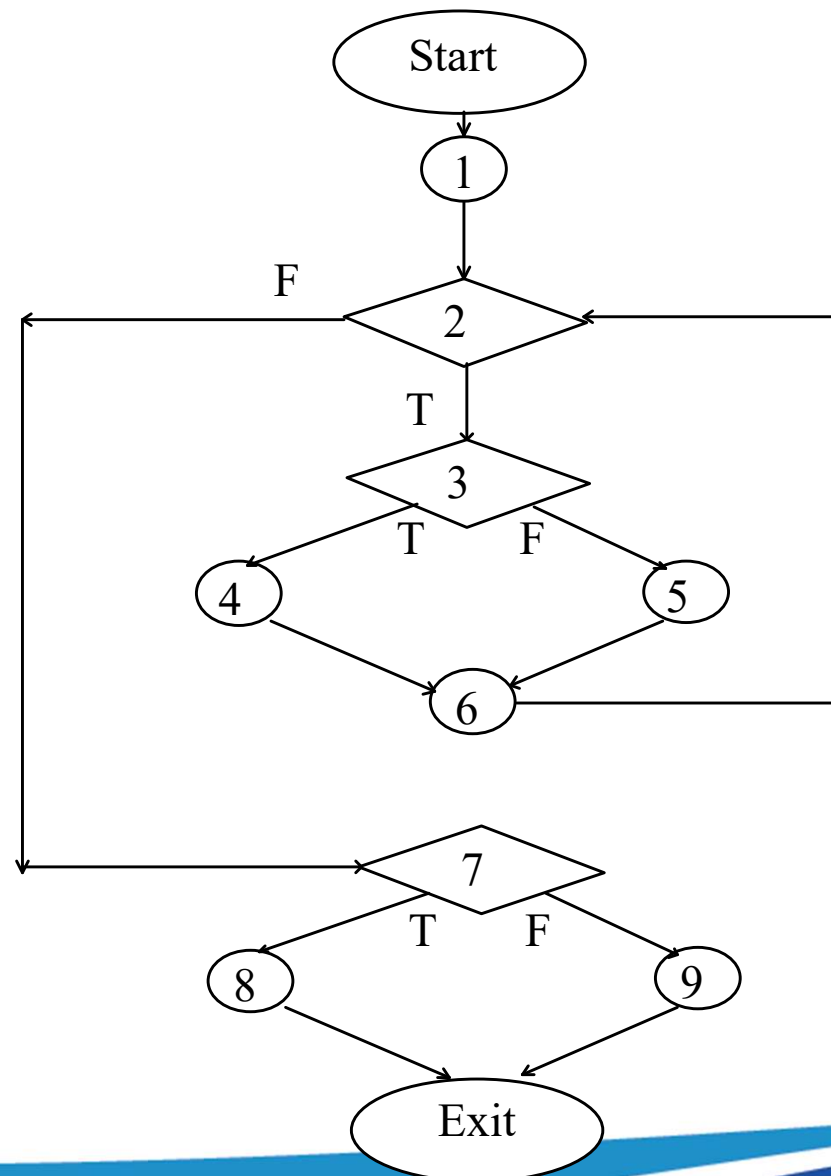
5

6

8

9

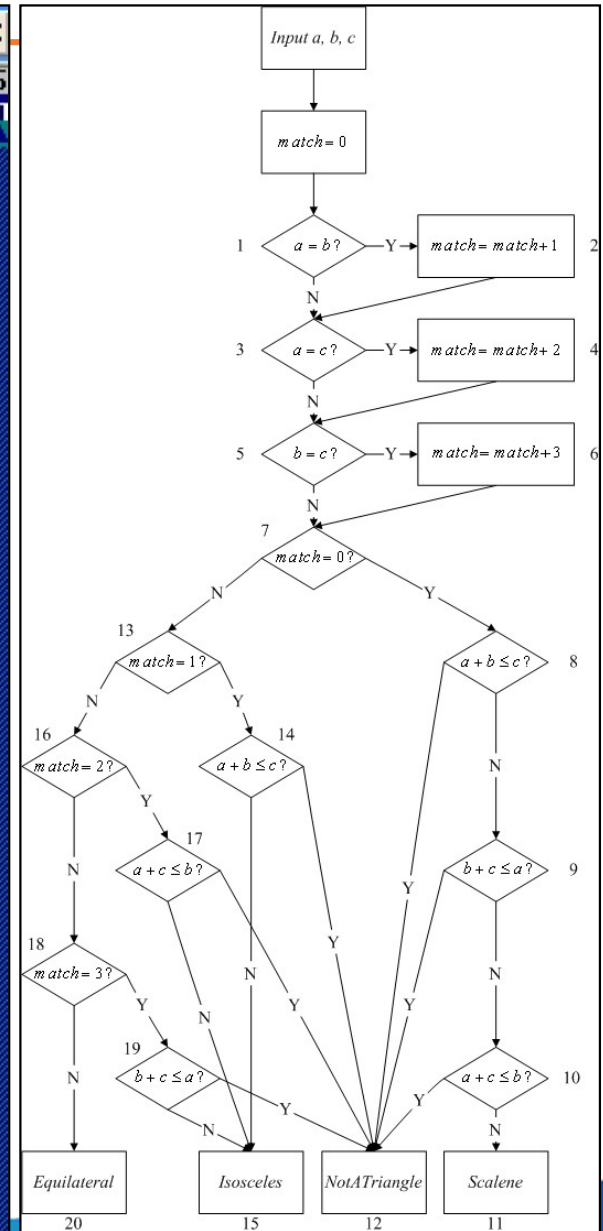
Sơ đồ khối





Đồ thị chương trình

```
Free Pascal IDE
File Edit Search Run Compile Debug Tools Options Window Help 14:26:25
C:\ECE453\one.pas
PROGRAM triangle1 <input, output>;
VAR a, b, c, match : INTEGER;
BEGIN
  writeln('Enter 3 integers which are sides of a triangle');
  readln(a,b,c);
  writeln('Side A is ',a);
  writeln('Side B is ',b);
  writeln('Side C is ',c);
  match := 0;
  IF a = b
  THEN match := match + 1;
  IF a = c
  THEN match := match + 2;
  IF b = c
  THEN match := match + 3;
  IF match = 0
  THEN IF <a+b> <= c
       THEN writeln('Not a Triangle')
       ELSE IF <b+c> <= a
            THEN writeln('Not a Triangle')
            ELSE IF <a+c> <= b
                 THEN writeln('Not a Triagle')
                 ELSE writeln('Triangle is Scalene')
  ELSE IF match = 1
  THEN IF <a+b> <= c
       THEN writeln('Not a Triangle')
       ELSE writeln('Triangle is Isosceles')
  ELSE IF match = 2
  THEN IF <a+c> <= b
       THEN writeln('Not a Triangle')
       ELSE writeln('Triangle is Isosceles')
  ELSE IF match = 3
  THEN IF <b+c> <= a
       THEN writeln('Not a Triangle')
       ELSE writeln('Triangle is Isosceles')
  ELSE writeln('Triangle is Equilateral');
  readln();
END.
```





Đồ thị chương trình và đường đi?

```
1.  x = z+5
2.  z = 4*3 - y
3.  if(x > z) goto A;
4.  for( u=0; u < x; u++) {
5.      z = z+1;
6.  };
7.  A: y = z + k
```




Đường thi hành

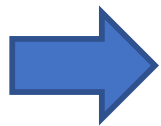
- Đường thi hành *Execution path*) : là 1 kịch bản thi hành đơn vị phần mềm tương ứng, là danh sách có thứ tự các câu lệnh được thi hành ứng với 1 lần chạy của đơn vị phần mềm bắt đầu từ điểm nhập của đơn vị phần mềm đến điểm kết thúc của đơn vị phần mềm.
- Mỗi đối tượng được kiểm thử thường có từ 1 đến N đường thi hành khác nhau.



Đường thi hành (2)

- Đường thi hành có thể rất dài

```
for (i=1; i<=1000; i++)  
    for (j=1; j<=1000; j++)  
        for (k=1; k<=1000; k++)  
            doSomethingWith(i,j,k);
```



*Chỉ có 1 đường thi hành duy nhất nhưng rất dài:
 $1000 \times 1000 \times 1000 = 1$ tỷ lần gọi hàm `doSomethingWith(i,j,k)`*



Đường thi hành (3)

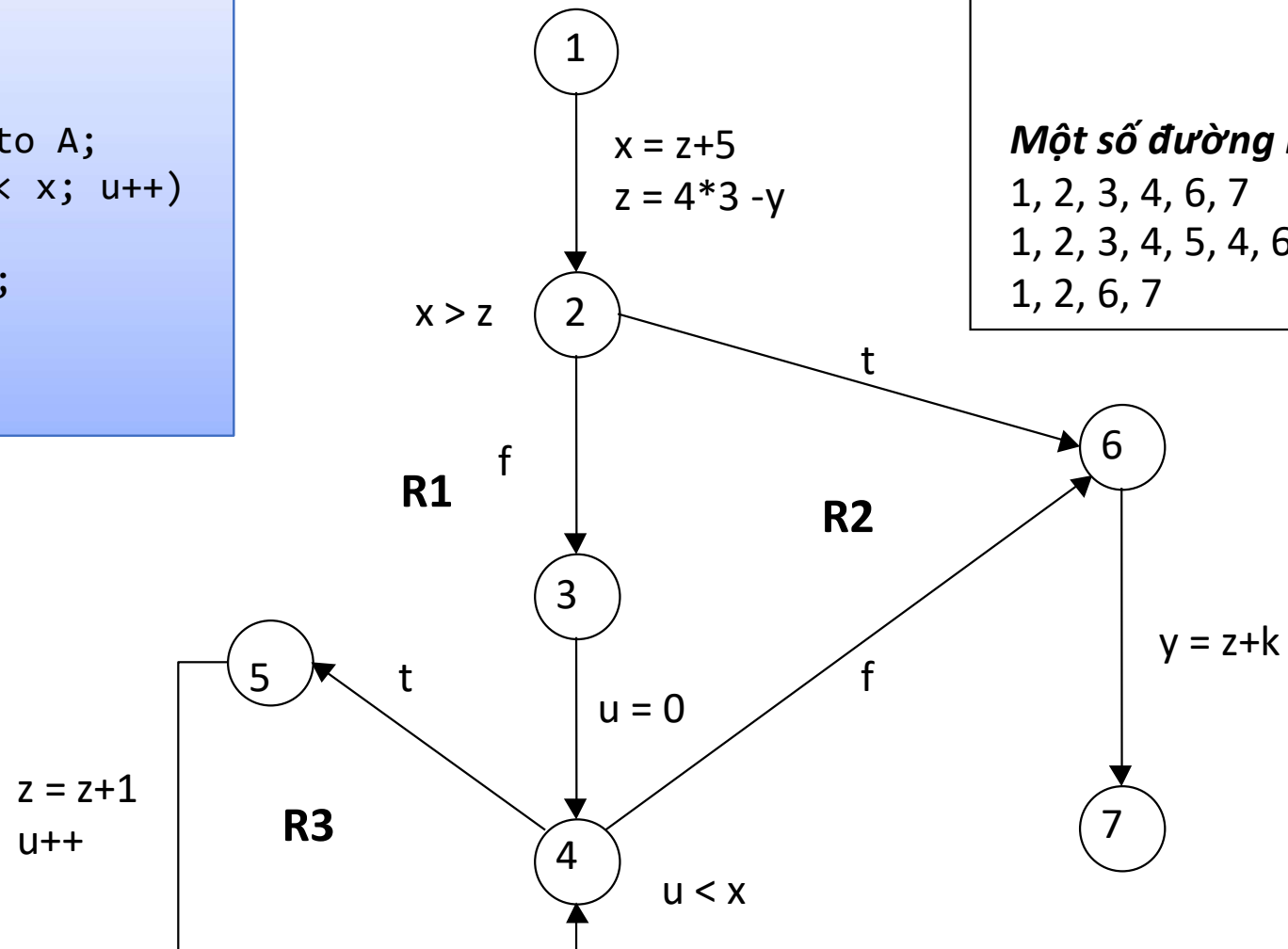
- Có thể có số đường thi hành rất lớn

```
if (c1) s11 else s12;  
if (c2) s21 else s22;  
if (c3) s31 else s32;  
...  
if (c32) s321 else s322;
```

có $2^{32} = 4$ tỉ đường thi hành khác nhau.

Các đường kiểm thử

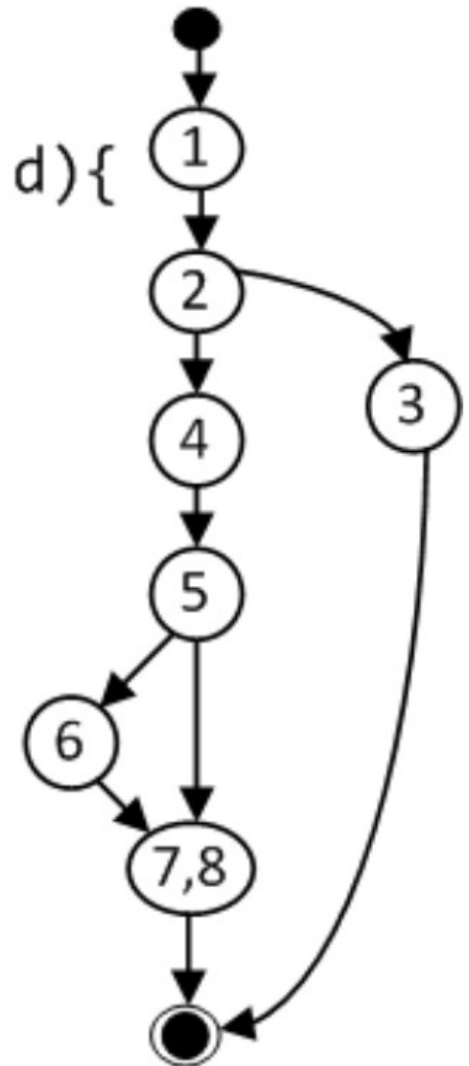
```
1. x = z+5
2. z = 4*3 - y
3. if(x > z) goto A;
4. for( u=0; u < x; u++)
{
5.     z = z+1;
6. };
7. A: y = z + k
```





Ví dụ 2

```
float foo(int a, int b, int c, int d){  
1.  float e;  
2.  if (a==0)  
3.      return 0;  
4.  int x = 0;  
5.  if ((a==b) || (c==d))  
6.      x = 1;  
7.  e = 1/x;  
8.  return e;  
}
```



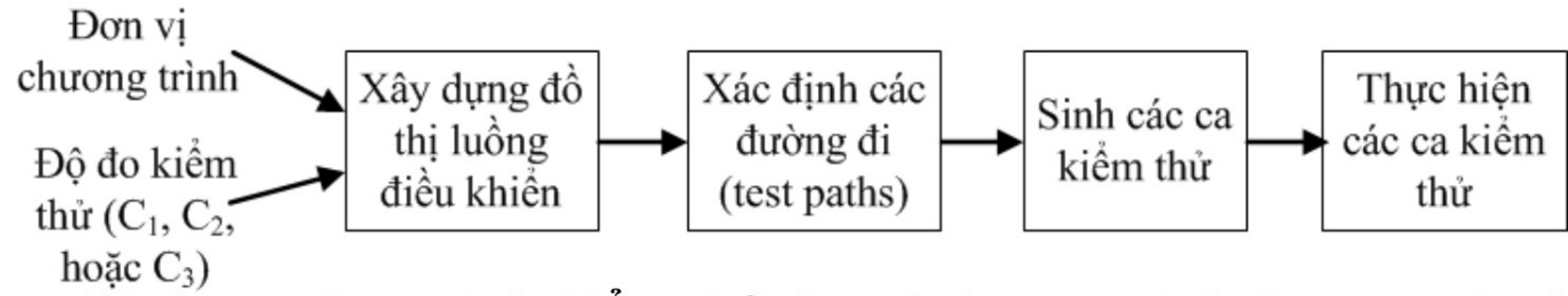


Độ phức tạp Cyclomatic

- Độ phức tạp Cyclomatic của đồ thị dòng điều khiển: $C = V(G)$
 - $V(G) = E - N + 2$, E là số cung, N là số nút
 - $V(G) = P + 1$, P là số nút quyết định với đồ thị dòng điều khiển nhị phân (chỉ có 2 cung T-F ở mỗi nút)
- Độ phức tạp ***C chính là số đường thi hành tuyến tính độc lập của đơn vị phần mềm cần kiểm thử***



Quy trình kiểm thử đơn vị dựa trên độ đo





Quy trình kiểm thử hộp trắng tổng quát

1. Xây dựng đồ thị dòng điều khiển
2. Tính độ phức tạp C của đồ thị
3. Xác định C đường thi hành tuyến tính độc lập cơ bản cần kiểm thử
4. Tạo từng test case cho từng đường thi hành trên
5. Thực hiện kiểm thử từng test case
6. So sánh kết quả và báo cáo



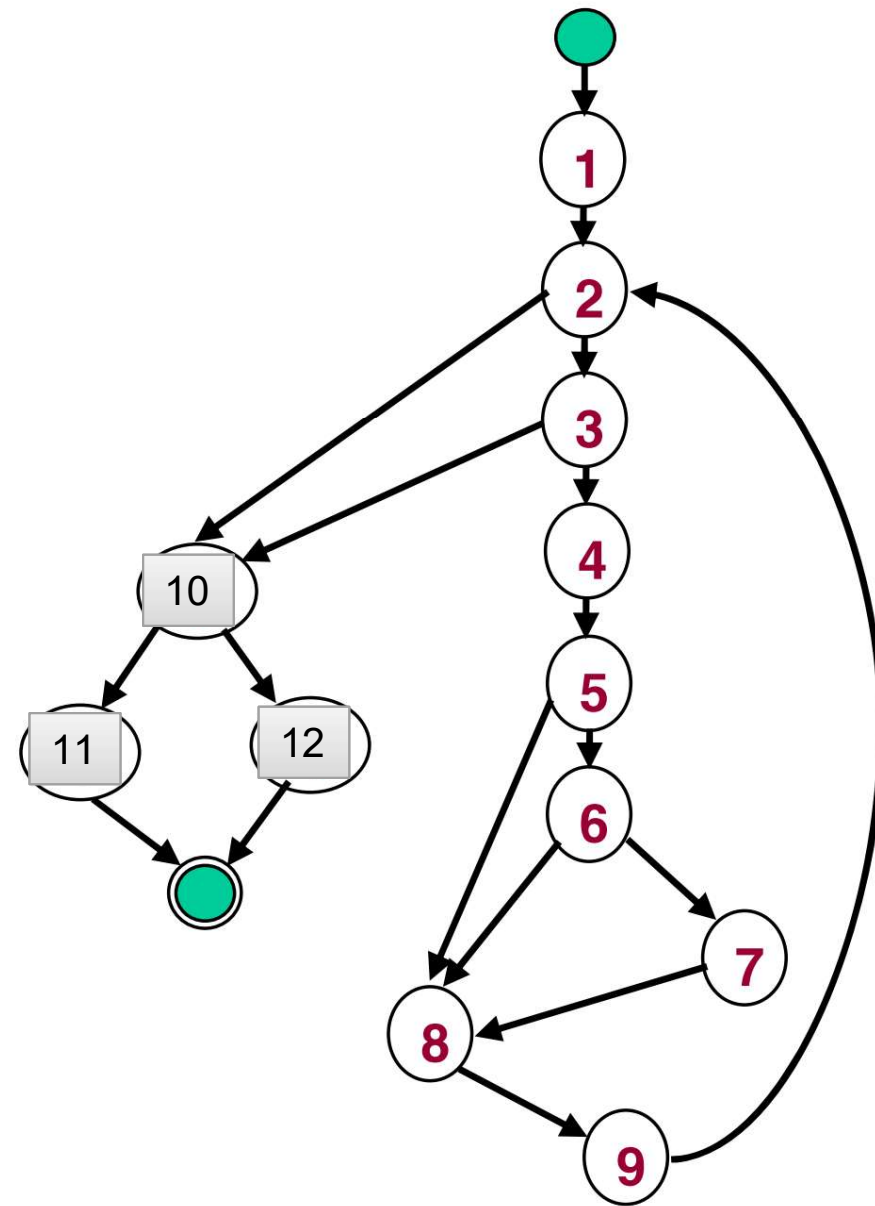
Quy trình xác định các đường tuyến tính độc lập

- Xác định đường tuyến tính đầu tiên bằng cách đi dọc theo nhánh bên trái nhất của các nút quyết định. Chọn đường này là pilot.
- Dựa trên đường pilot, thay đổi cung xuất của nút quyết định đầu tiên và cố gắng giữ lại maximum phần còn lại
- Dựa trên đường pilot, thay đổi cung xuất của nút quyết định thứ hai và cố gắng giữ lại maximum phần còn lại
- Tiếp tục thay đổi cung xuất của từng nút quyết định trên đường pilot để xác định đường thứ 4,5, ... cho đến khi không còn nút quyết định nào trên đường pilot.
- Lặp chọn tuần tự từng đường tìm được làm pilot để xác định các đường mới xung quanh nó y như các bước 2,3,4 cho đến khi không tìm được đường tuyến tính độc lập nào nữa (khi đủ số C).


```

double average(double value[], double min,
               double max, int& tcnt, int& vcnt) {
    double sum = 0;
    int i = 1;
    tcnt = vcnt = 0;
    while (value[i] <> -999 && tcnt < 100) {
        tcnt++;
        if (min <= value[i] && value[i] <= max) {
            sum += value[i];
            vcnt++;
        }
        i++;
    }
    if (vcnt > 0) return sum/vcnt;
    return -999;
}

```



5 nút quyết định nhị phân: $C = 5 + 1 = 6$

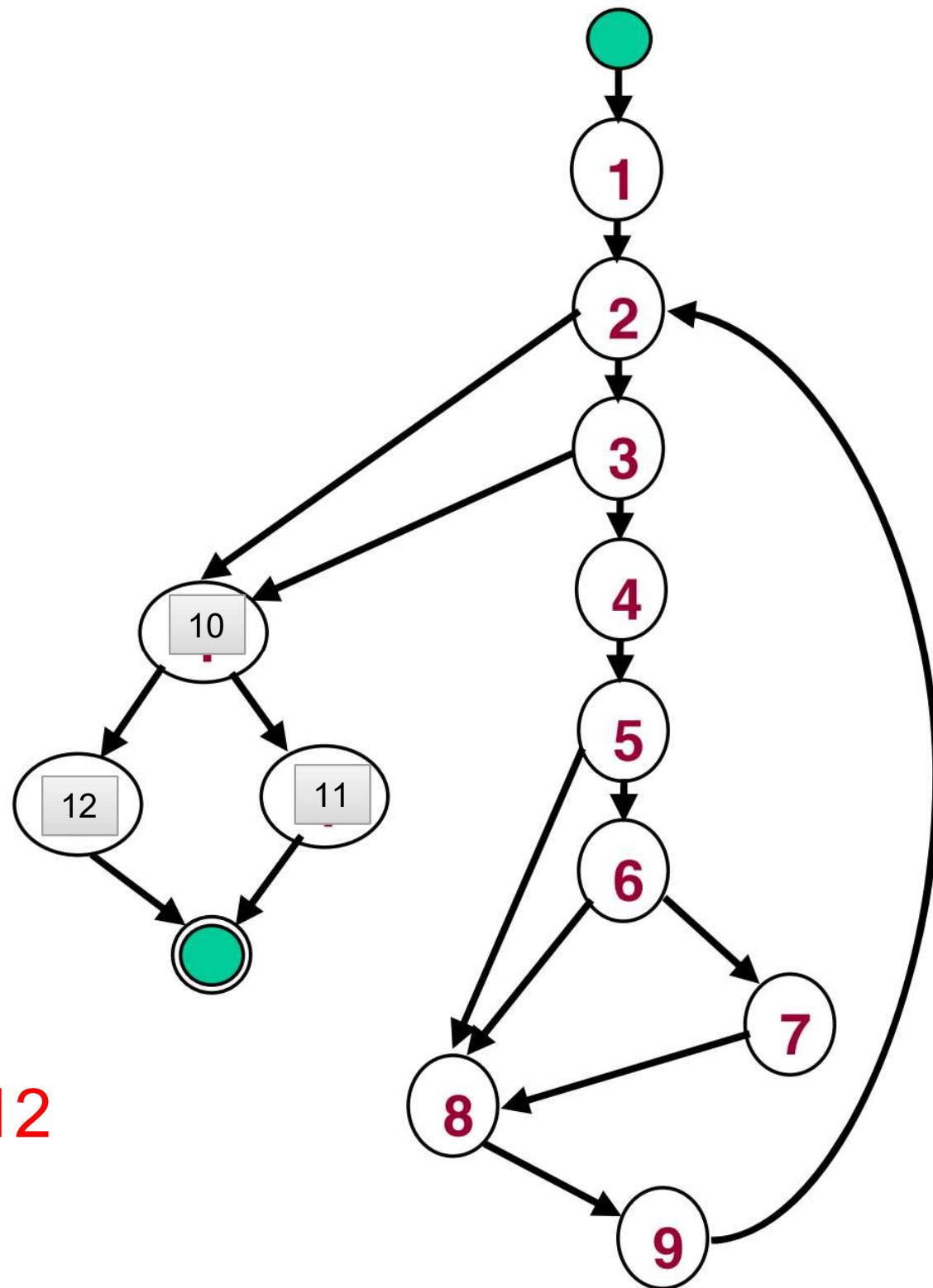


$C=6$

6 đường thi hành tuyến
tính độc lập cơ bản:

1. $1 \rightarrow 2 \rightarrow 10 \rightarrow 11$
2. $1 \rightarrow 2 \rightarrow 3 \rightarrow 10 \rightarrow 11$
3. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 8 \rightarrow 9$
4. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 8 \rightarrow 9$
5. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9$
6. $1 \rightarrow 2 \rightarrow 10 \rightarrow 12$

$9 \rightarrow 2/3 \rightarrow 10 \rightarrow 11|12$





Độ đo kiểm thử

- Đo mức độ bao phủ chương trình của một tập ca kiểm thử cho trước
- Mức độ bao phủ của một bộ kiểm thử (tập các ca kiểm thử) được đo bằng tỷ lệ các thành phần thực sự được kiểm thử so với tổng thể sau khi đã thực hiện các ca kiểm thử
- Độ bao phủ càng lớn thì độ tin cậy của bộ kiểm thử càng cao
- Mục tiêu: số ca kiểm thử tối thiểu nhưng đạt độ bao phủ tối đa.



Ba độ đo kiểm thử phổ biến (C1, C2, C3)

- Độ đo kiểm thử cấp 1 (C1): mỗi câu lệnh được thực hiện ít nhất một lần sau khi chạy bộ kiểm thử.
- Độ đo kiểm thử cấp 2 (C2): mỗi điểm quyết định trong đồ thị dòng điều khiển đều được thực hiện ít nhất một lần cho cả hai nhánh đúng và sai.
- Độ đo kiểm thử cấp 3 (C3): với các điều kiện phức tạp (chứa nhiều điều kiện cơ bản), mỗi điều kiện con cần thực hiện ít nhất một lần cho cả hai nhánh đúng và sai.



- C1: đi qua mỗi đỉnh 1 lần, có rẽ nhánh thì ko nhất thiết phải qua hết nhánh True/False
- C2: phải đi qua cả nhánh True/False, nếu có lệnh if mà có nhiều biểu thức điều kiện thì ko nhất thiết phải đi qua hết các biểu thức con
- C3: Đi qua hết các điều kiện của của các biểu thức con trong if



Độ đo kiểm thử cấp 1 (C1)

- Mỗi câu lệnh được thực hiện ít nhất một lần sau khi chạy bộ kiểm thử.

ID	Inputs	EO	RO	Note
tc1	0, 1, 2, 3	0		
tc2	1, 1, 2, 3	1		

```
float foo(int a, int b, int c, int d){  
1.  float e;  
2.  if (a==0)  
3.      return 0;  
4.  int x = 0;  
5.  if ((a==b) || (c==d))  
6.      x = 1;  
7.  e = 1/x;  
8.  return e;  
}
```



Độ đo kiểm thử cấp 2 (C2)

- Mỗi điểm quyết định trong đồ thị dòng điều khiển đều được thực hiện ít nhất một lần cho cả hai nhánh đúng và sai

ID	Inputs	EO	RO	Note
tc1	0, 1, 2, 3	0		
tc2	1, 1, 2, 3	1		
tc3	1, 2, 1, 2	Lỗi chia cho 0		

```
float foo(int a, int b, int c, int d){  
1.  float e;  
2.  if (a==0)  
3.      return 0;  
4.  int x = 0;  
5.  if ((a==b) || (c==d))  
6.      x = 1;  
7.  e = 1/x;  
8.  return e;  
}
```




Độ đo kiểm thử cấp 3 (C3)

- Với các điều kiện phức tạp (chứa nhiều điều kiện cơ bản), mỗi điều kiện còn cần thực hiện ít nhất một lần cho cả hai nhánh đúng và sai.

ID	Inputs	EO	RO	Note
tc1	0, 1, 2, 3	0		
tc2	1, 1, 2, 3	1		
tc3	1, 2, 1, 2	Lỗi chia cho 0		
tc4	1, 2, 1, 1	1		

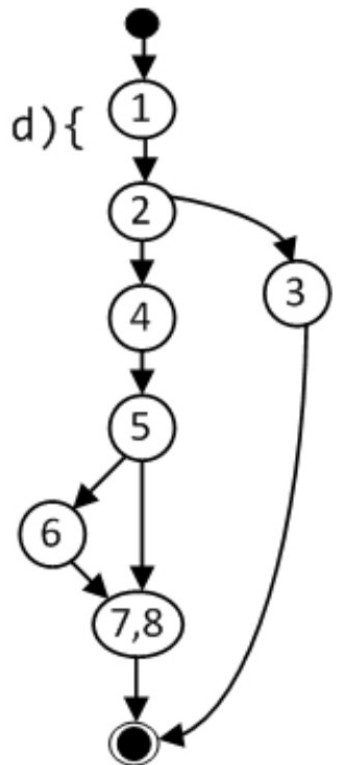
```
float foo(int a, int b, int c, int d){  
1.  float e;  
2.  if (a==0)  
3.      return 0;  
4.  int x = 0;  
5.  if ((a==b) || (c==d))  
6.      x = 1;  
7.  e = 1/x;  
8.  return e;  
}
```




Ví dụ các ca kiểm thử để đạt C1

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2(F); 4; 5(T); 6; 7,8	2, 2, 3, 5	1		
tc2	1; 2(T); 3	0, 3, 2, 7	0		

```
float foo(int a, int b, int c, int d){  
1.  float e;  
2.  if (a==0)  
3.      return 0;  
4.  int x = 0;  
5.  if ((a==b) || (c==d))  
6.      x = 1;  
7.  e = 1/x;  
8.  return e;  
}
```

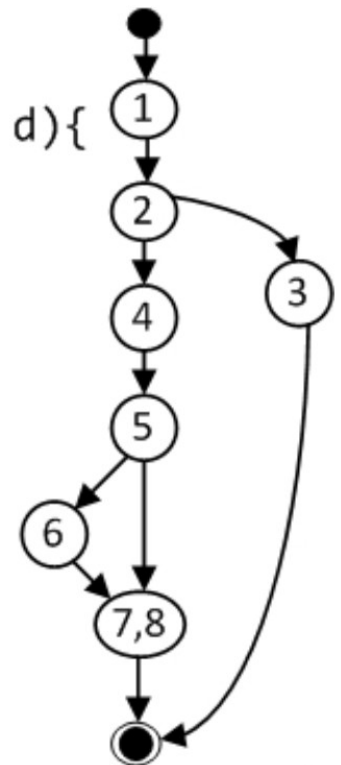




Ví dụ các ca kiểm thử để đạt C2

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2(F); 4; 5(T); 6; 7,8	2,2,3, 5	1		
tc2	1; 2(T); 3	0,3,2,7	0		
tc3	1; 2(F); 4; 5(F); 7,8	2,3,4,5	lỗi chia cho 0		

```
float foo(int a, int b, int c, int d){  
1.  float e;  
2.  if (a==0)  
3.    return 0;  
4.  int x = 0;  
5.  if ((a==b) || (c==d))  
6.    x = 1;  
7.  e = 1/x;  
8.  return e;  
}
```

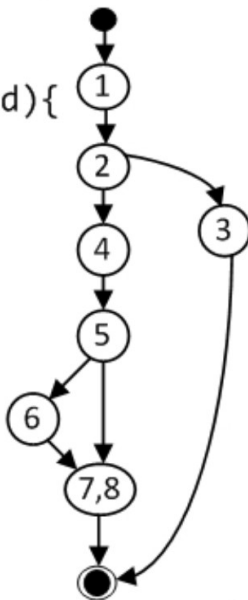




Ví dụ các ca kiểm thử để đạt C3

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2(F); 4; 5c1(T); 6; 7,8	0, 2, 3, 5	0		
tc2	1; 2(F); 4; 5c1(F); 5c2(T); 6; 7,8	2, 2, 2, 7	1		
tc3	1; 2(F); 4; 5c1(F); 5c2(F); 7,8	2,3,4,5	lỗi chia cho 0		
tc4	1; 2(T); 3	2,3,4,4	1		

```
float foo(int a, int b, int c, int d){  
1. float e;  
2. if (a==0)  
3.     return 0;  
4. int x = 0;  
5. if ((a==b) || (c==d))  
6.     x = 1;  
7. e = 1/x;  
8. return e;  
}
```





Ví dụ 3.2.1

Dựa vào đoạn code dưới đây: vẽ đồ thị luồng chương trình, xác định C, xác định các đường thi hành

```
function {} tinh_tong(int n)
{
    int tong=0, i;
    int tong1 = 0, tong2 = 0;
    if (n<1)
        return 0;
    for (i = 2; i<=n; i++) {
        if ( i%2 = 0 && i>= 10)
            tong1 += i;
        else
            tong2 +=i;
        tong+=i;
    }
    return {tong, tong1, tong2};
}
```



Ví dụ C1,C2,C3

Dựa trên đoạn code sau, xác định các đường đi tương ứng với độ đo C1,C2,C3.

```
function {} tinh_tong(int n)
{
    int tong=0, i;
    int tong1 = 0, tong2 = 0;
    if (n<1)
        return 0;
    for (i = 2; i<=n; i++) {
        if ( i%2 = 0 && i>= 10)
            tong1 += i;
        else
            tong2 +=i;
        tong+=i;
    }
    return {tong, tong1, tong2};
}
```



Bài tập: Xây dựng test case dựa trên đồ thị dòng điều khiển

Hàm: Giải phương trình bậc 2

- Xây dựng đồ thị dòng điều khiển
- Tính độ phức tạp C của đồ thị và xác định đường thi hành
- Xác định các test case tương ứng C1,C2,C3



GPTB2

```
int [] PGTB2(int a, int b, int c){  
1.     float x=0,x1, x2, detal;  
2.     if (a==0) return [0];  
3.     delta = b*b - 4 * a *c ;  
4.     if (detal <0)  
5.     {     x= [1];  
6.           return [x];  
7.     }  
8.     if (delta ==0)  
9.     {     x = (-b/(2*a));  
10.          return [x];  
11.     }  
12.     x1 = (-b + sqrt (detal)) / (2*a);  
13.     x2 = (-b - sqrt (detal)) / (2*a);  
14.     return [x1,x2];  
}
```



Kiểm thử vòng lặp

- Lệnh lặp đơn giản: đơn vị chương trình chứa đúng một vòng lặp
- Lệnh lặp liên tiếp: đơn vị chương trình chứa các lệnh lặp kế tiếp nhau
- Lệnh lặp lồng nhau: đơn vị chương trình chứa các vòng lặp chứa các lệnh lặp khác

Ví dụ: các thuật toán sắp xếp (3 đơn giản, 3 nâng cao)



Vd: vòng lặp đơn

- Function tong(int n)
- {
 1. int tong=1, i;
 2. if (n<1)
 3. return 0;
 4. for (i = 2; i<=n; i++)
 5. tong += i;
 6. return tong;
- }



Vd: Lệnh lặp liên kè

- Function tong_le(int n)
- {
 1. int tong=0, i, tongle=0;
 2. if (n<1)
 3. return 0;
 4. for (i = 1; i<=n; i++)
 5. tong += i;
 6. for (i = 1; i<=n; i++) (
 7. if (i%2 !=0) tongle += i;
 8. }
 9. return tong;
- }







Chương trình chỉ có lệnh lặp đơn giản

Cần số ca kiểm thử tương ứng với các trường hợp:

- Vòng lặp thực hiện 0 lần
- Vòng lặp thực hiện 1 lần
- Vòng lặp thực hiện 2 lần ($2 < k < n-1$, n là số lần lặp tối đa của vòng lặp)
- Vòng lặp thực hiện k lần
- Vòng lặp thực hiện $n-1$ lần
- Vòng lặp thực hiện n lần
- Vòng lặp thực hiện $n+1$ lần



Bài tập vận dụng: C1, C2, C3, kiểm thử vòng lặp

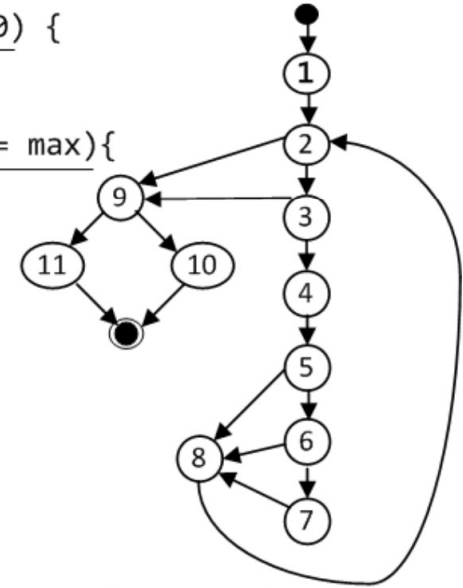
	double average(double value[], double min, double max, int& tcnt, int& vcnt) {	
1	double sum = 0;	4.1
2	int i = 1;	4.2
3	tcnt = vcnt = 0;	
4	while (value[i] <> -999 && tcnt < 100) {	6.1
5	tcnt++;	
6	if (min <= value[i] && value[i] <= max) {	6.2
7	sum += value[i];	
8	vcnt ++;	
	}	
9	i++;	
	}	
10, 11	if (vcnt > 0) return sum/vcnt;	
12	return -999;	
	}	



Ví dụ

*Các ca kiểm thử
cho hàm average
với độ đo C3:*

```
double average(double value[], double min, double
max, int& tcnt, int& vcnt) {
double sum = 0;
int i = 1;
tcnt = vcnt = 0;
while (value[i] <> -999 && tcnt <100) {
    tcnt++;
    if (min<=value[i] && value[i]<= max){
        sum += value[i];
        vcnt ++;
    }
    i++;
} //end while
if (vcnt > 0)
    return sum/vcnt;
return -999;
} //end
```

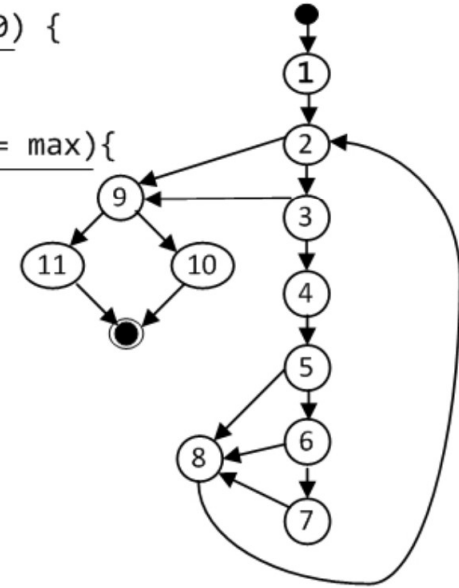


ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2(F); 9(T); 10				tc6
tc2	1; 2(F); 9(F); 11	[-999,...], 1, 2	-999		
tc3	1; 2(T); 3(F); 9(T); 10				tc6
tc4	2(T); 3(T); 4; 5(F); 8; 2(F); 9(F); 11	[0,-999], 1, 2	-999		
tc5	1; 2(T); 3(T); 4; 5(T); 6(F); 8; 2(F); 9(F); 11	[3,-999], 1, 2	-999		
tc6	1; 2(T); 3(T); 4; 5(T); 6(T); 7; 8; 2(F); 9(T); 10	[1,-999], 1, 2	1		



Các ca kiểm thử cho hàm average với kiểm thử vòng lặp:

```
double average(double value[], double min, double  
max, int& tcnt, int& vcnt) {  
    double sum = 0; | 1  
    int i = 1;  
    tcnt = vcnt = 0;  
    while (value[i] <> -999 && tcnt < 100) {  
        | 2  
        tcnt++; | 4  
        if (min <= value[i] && value[i] <= max) {  
            | 5  
            sum += value[i]; | 7  
            vcnt++;  
        }  
        i++; | 8  
    } //end while  
    if (vcnt > 0) | 9  
        return sum/vcnt; | 10  
    return -999; | 11  
} //end
```



ID	Lần lặp	Inputs	EO	RO	Note
tcl0	0	[-999, ...], 1, 2	-999		
tcl1	1	[1,-999], 1, 2	1		
tcl2	2	[1,2,-999], 1, 2	1.5		
tclk	5	[1,2,3,4,5,-999], 1, 10	3		
tcl(n-1)	99	[1,2,...,99,-999], 1, 100	50		
tcln	100	[1,2,...,100], 1, 2	50.5		
tcl(n+1)					



Kiểm thử vòng lặp lồng nhau

- Kiểm thử tuần tự từng vòng lặp từ trong ra ngoài theo đề nghị sau:
 - Kiểm thử vòng lặp trong cùng: cho các vòng lặp ngoài chạy với giá trị min, kiểm thử vòng lặp trong cùng với 7 test cases đã hướng dẫn.
 - Kiểm thử từng vòng lặp còn lại: cho các vòng ngoài nó chạy với giá trị min, các vòng bên trong nó chạy với các giá trị điển hình, kiểm thử nó bằng 7 test cases đã hướng dẫn.



Kiểm thử các vòng lặp liên kề

- Kiểm thử tuần tự từng vòng lặp từ trên xuống, mỗi vòng lặp kiểm thử bằng 7 test cases như đã hướng dẫn.



Nội dung

1. Kiểm thử chức năng
 - Kiểm thử giá trị biên
 - Kiểm thử lớp tương đương
 - Kiểm thử bảng quyết định
2. Kiểm thử cấu trúc
 - Tổng quan
 - Kiểm thử luồng điều khiển
 - **Kiểm thử luồng dữ liệu**



Tổng quan kiểm thử dòng dữ liệu

- Kiểm thử đời sống của từng biến dữ liệu trong từng đường thi hành của chương trình
- Là công cụ mạnh để phát hiện việc dùng không hợp lý các biến do lỗi khi viết chương trình
 - Phát biểu gán hay nhập dữ liệu vào biến không đúng
 - Thiếu định nghĩa biến trước khi dùng



Chu kỳ sống của biến

- 3 bước theo trình tự: được tạo ra, được dùng và được xóa đi
- Chỉ có những lệnh nằm trong khu vực truy xuất biến mới có thể truy xuất/xử lý biến
 - Biến toàn cục
 - Biến cục bộ

```
int x, y;
```

```
void func1() { //thân hàm
```

```
    int x;    // định nghĩa biến x mới cục bộ trong hàm
```

```
    ...;      // mỗi lần truy xuất x là x cục bộ trong hàm
```

```
    {        // khối lệnh bên trong bắt đầu
```

```
        int y; // định nghĩa biến y mới cục bộ trong lệnh phức hợp
```

```
        ...;   //mỗi lần truy xuất y là y cục bộ trong lệnh phức hợp
```

```
    }        // y bên trong tự động bị xóa
```

```
    ...;     //truy xuất y ngoài cùng, x cục bộ trong hàm
```

```
} // x cục bộ trong hàm bị xóa tự động
```



Phân tích đời sống của biến

- Các lệnh truy xuất một biến thông qua một trong ba hành động sau:
 - d: định nghĩa biến, gán giá trị xác định cho biến (bao gồm nhập dữ liệu cho biến)
 - u: tham khảo giá trị của biến (thường thông qua biểu thức)
 - k: hủy (xóa) bỏ biến
- Ký hiệu ~ là miêu tả trạng thái ở đó biến chưa tồn tại:
 - ~d: biến chưa tồn tại rồi được định nghĩa
 - ~u: biến chưa tồn tại rồi được dùng (vậy lấy giá trị nào?)
 - ~k: biến chưa tồn tại rồi bị hủy (???)



Phân tích đời sống của biến (2)

- Kết hợp các hoạt động xử lý biến khác nhau:
 - dd: biến được định nghĩa rồi, định nghĩa nữa (hơi lạ nhưng chấp nhận được)
 - du: biến được định nghĩa rồi, được dùng (bình thường)
 - dk: biến được định nghĩa rồi, được hủy (hơi lạ nhưng chấp nhận được)
 - ud: biến được dùng rồi được định nghĩa giá trị mới (hợp lý)
 - uu: biến được dùng rồi được dùng tiếp (hợp lý)
 - uk: biến được dùng rồi bị hủy (hợp lý)
 - kd: biến được dùng rồi được định nghĩa lại (chấp nhận được)
 - ku: biến bị hủy rồi được dùng lại (lỗi)
 - kk: biến bị xóa bỏ rồi bị xóa nữa (lỗi)



Đồ thị dòng dữ liệu

- Mô tả kịch bản đời sống của biển
- Xây dựng đồ thị dòng dữ liệu dựa trên đồ thị dòng điều khiển



Quy trình kiểm thử dòng dữ liệu

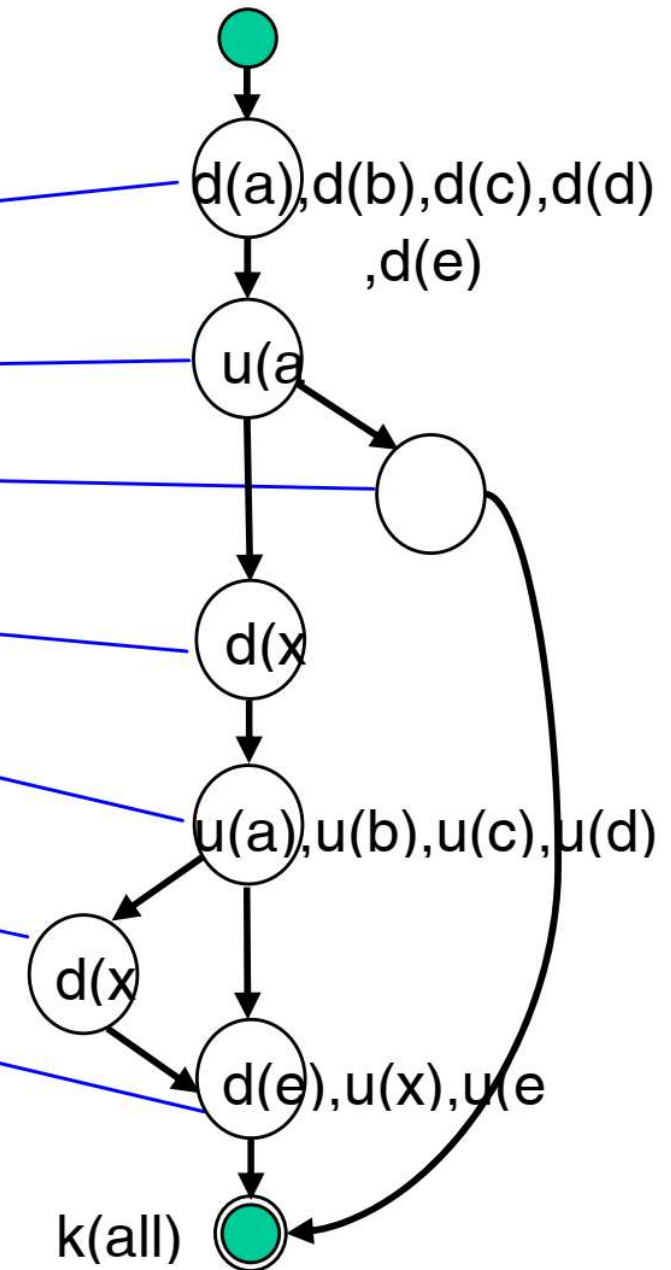
1. Xây dựng đồ thị dòng điều khiển của đơn vị phần mềm cần kiểm thử rồi chuyển thành đồ thị dòng dữ liệu
2. Tính độ phức tạp C của đồ thị
3. Xác định C đường thi hành tuyến tính độc lập
4. Lập kiểm thử đời sống từng biến dữ liệu
 - Mỗi biến có thể có tối đa C kịch bản đời sống khác nhau
 - Trong kịch bản đời sống của mỗi biến, kiểm thử xem có tồn tại cặp đôi hoạt động không bình thường nào không? Nếu có thì lập báo cáo

Ví dụ

```

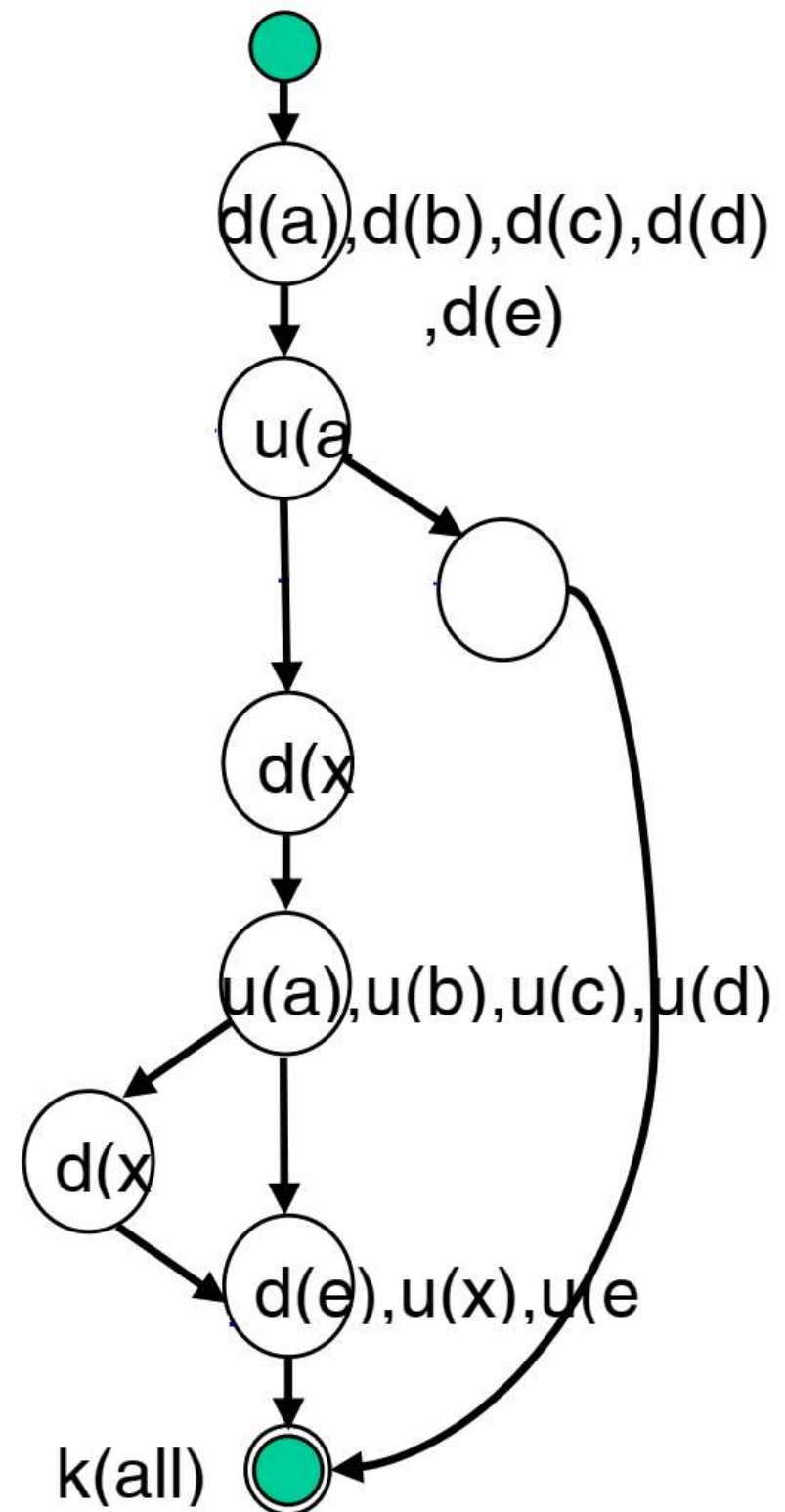
1. float foo(int a, int b, int c, int d) {
2.   float e;
3.   if (a==0)
4.     return 0;
5.   int x = 0;
6.   if ((a==b) || ((c==d) && bug(a)))
7.     x = 1;
8.   e = 1/x;
9.   return e;
10. }

```

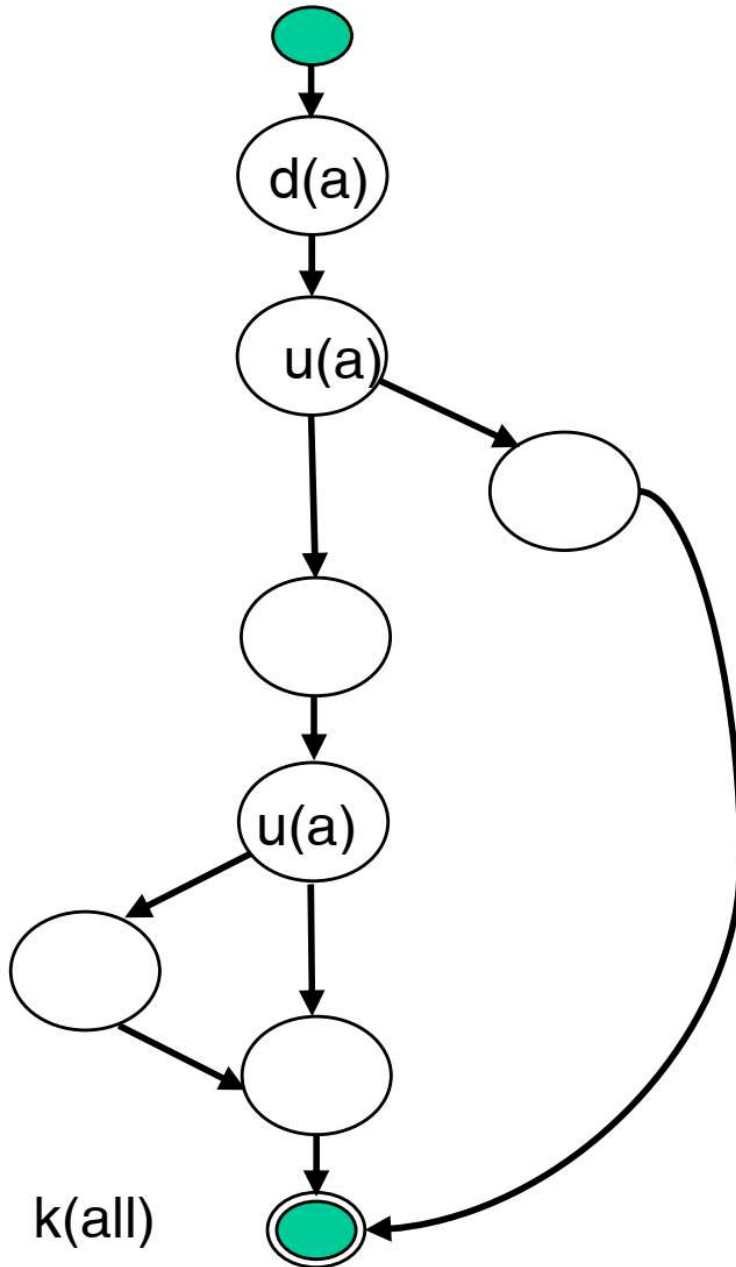


Ví dụ

- $C = 2 + 1 = 3$
- Có 4 biến đầu vào:
a, b, c, d
- Hai biến cục bộ: e, x
- Cần lập kiểm thử
đời sống từng biến:
a, b, c, d, e, x



Kiểm thử đời sống biến a

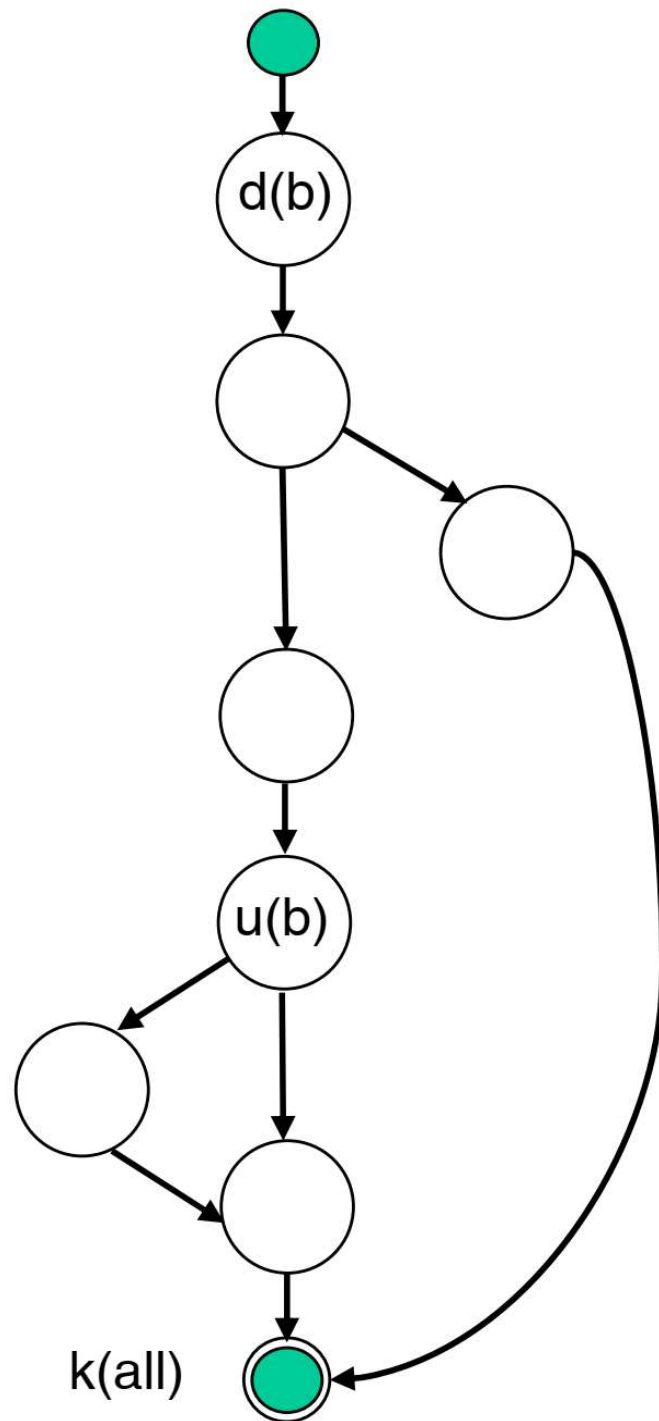


- Kịch bản 1 : $\sim duuk$
- Kịch bản 2 : $\sim duuk$ (giống kịch bản 1).
- Kịch bản 3 : $\sim duk$

Cả 3 kịch bản trên đều không chứa cặp đôi hoạt động nào bất thường cả.



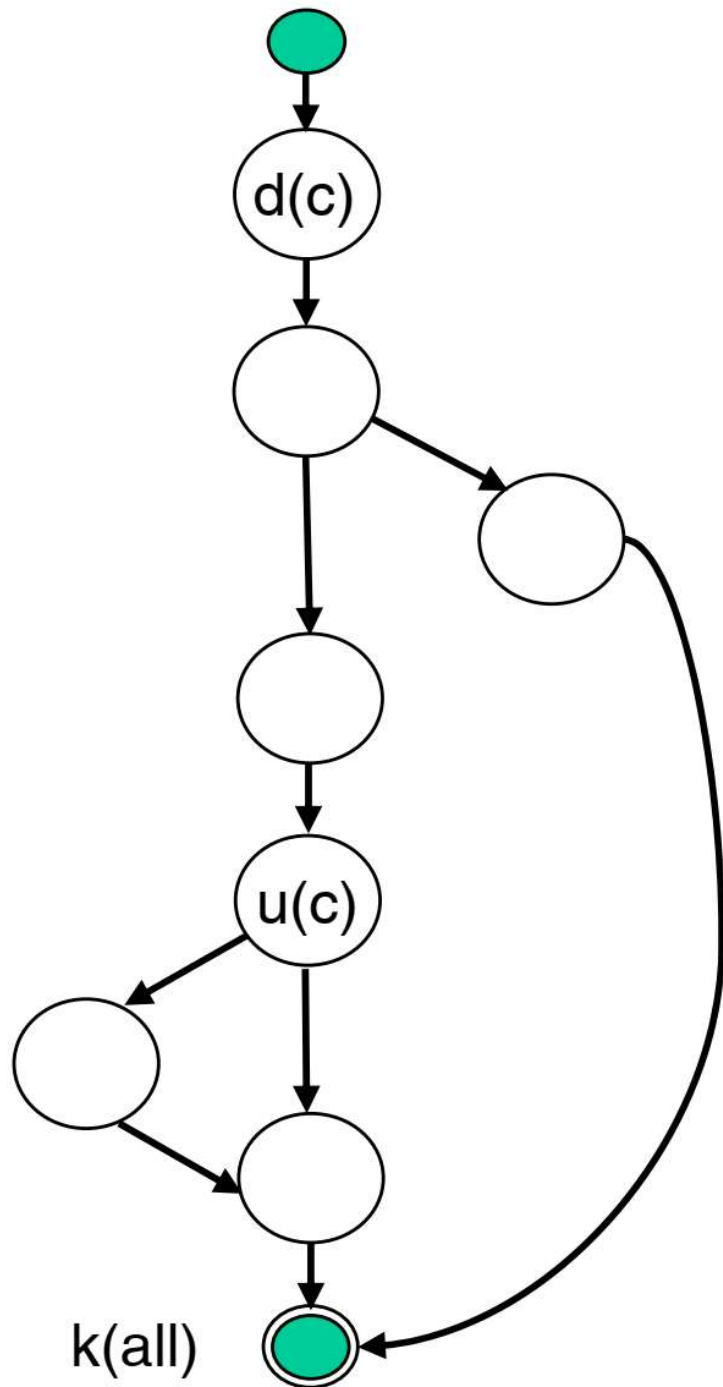
Kiểm thử đời sống biến b



- Kịch bản 1 : $\sim duk$
- Kịch bản 2 : $\sim duk$ (giống kịch bản 1).
- Kịch bản 3 : $\sim dk$

Cả 3 kịch bản trên đều không chứa cặp đôi hoạt động nào bất thường cả.

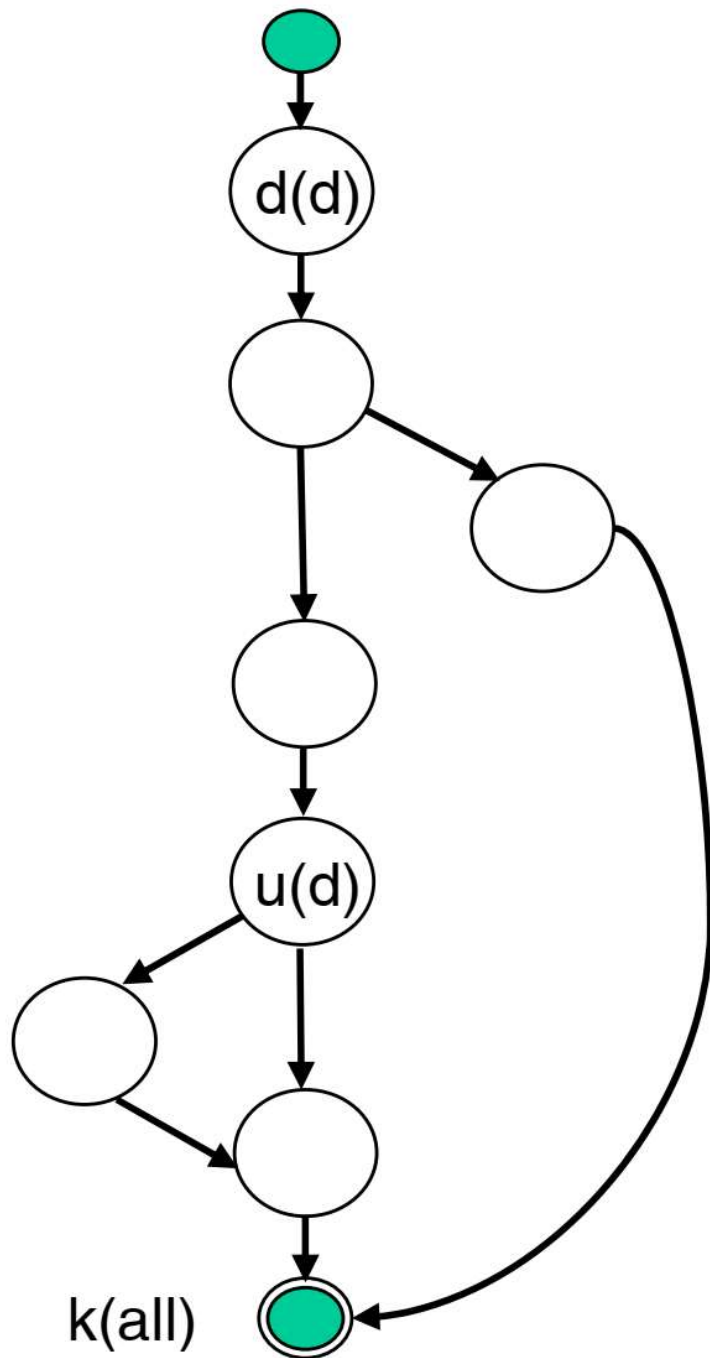
Kiểm thử đời sống biến c



- Kịch bản 1 : $\sim duk$
- Kịch bản 2 : $\sim duk$ (giống kịch bản 1).
- Kịch bản 3 : $\sim dk$

Cả 3 kịch bản trên đều không chứa cặp đôi hoạt động nào bất thường cả.

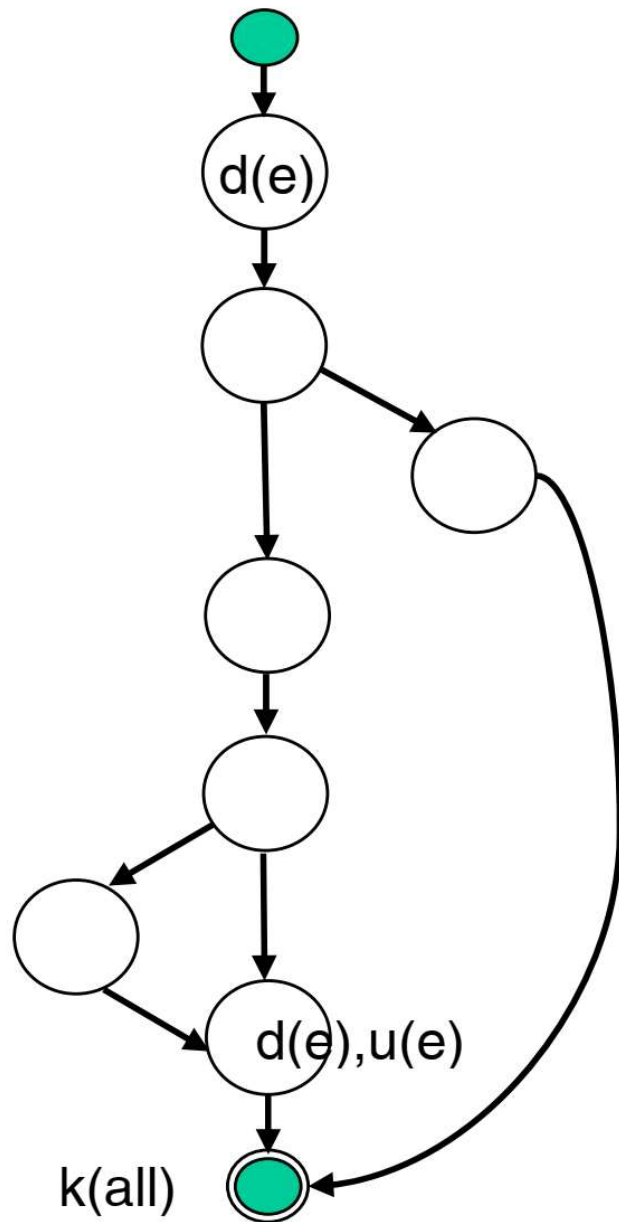
Kiểm thử đời sống biến d



- Kịch bản 1 : $\sim duk$
- Kịch bản 2 : $\sim duk$ (giống kịch bản 1).
- Kịch bản 3 : $\sim dk$

Cả 3 kịch bản trên đều không chứa cặp đôi hoạt động nào bất thường cả.

Kiểm thử đời sống biến e

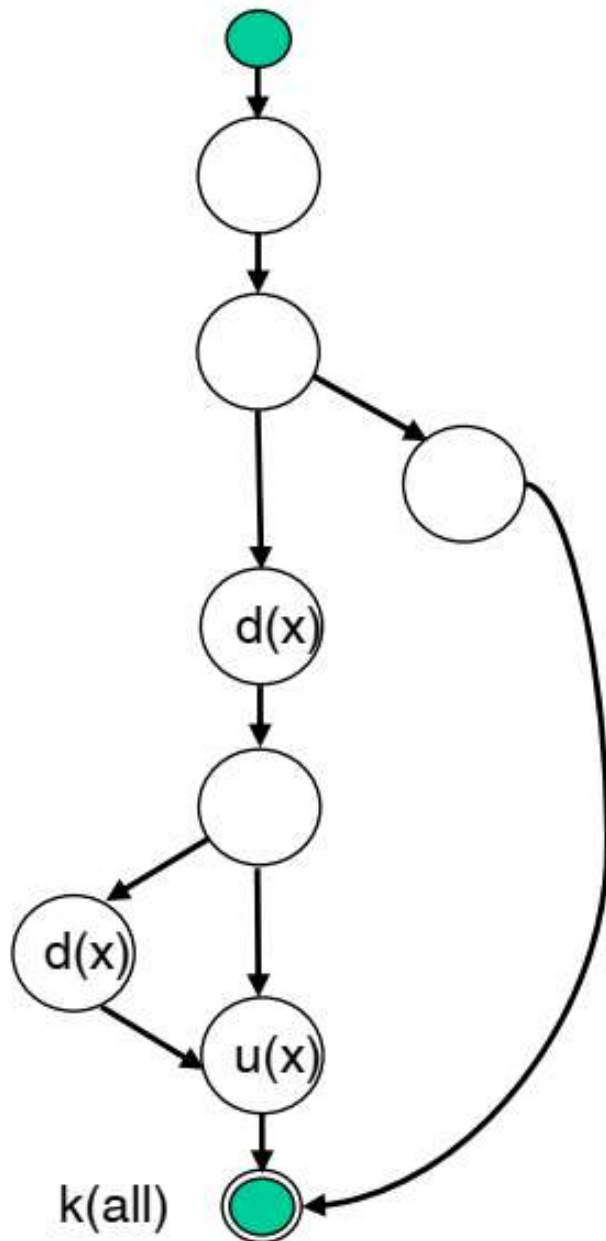


- Kịch bản 1 : $\sim dduk$
- Kịch bản 2 : $\sim dduk$ (giống kịch bản 1).
- Kịch bản 3 : $\sim dk$

Trong 3 kịch bản trên, kịch bản 1 & 2 có chứa cặp đôi dd bất thường nên cần tập trung chú ý kiểm tra xem có phải là lỗi không.



Kiểm thử đời sống biến x



- Kịch bản 1 : $\sim dduk$
- Kịch bản 2 : $\sim duk$
- Kịch bản 3 : $\sim$

Trong 3 kịch bản trên, chỉ có kịch bản 1 có chứa cặp đôi dd bất thường nên cần tập trung chú ý kiểm tra xem có phải là lỗi không.



Tổng kết

- Nên kết hợp áp dụng cả kiểm thử hộp đen và hộp trắng
- Các phương pháp kiểm thử hộp đen có thể bao phủ đường đi tốt nhưng thường có nhiều dư thừa
- Liệu chúng ta có thể có một độ đo để so sánh mức độ hiệu quả của một kỹ thuật hộp đen với một kỹ thuật hộp trắng?
 - Các phương pháp hộp đen được đo trên số ca kiểm thử tạo ra
 - Các phương pháp hộp trắng được đo trên mức độ bao phủ đường đi chúng đạt được



Bài tập

- Áp dụng các phương pháp thiết kế ca kiểm thử đã học vào các chương trình được giao viết ở chương 1.



Kiểm thử hộp trắng

- Dựa trên đoạn mã sau

+ Xây dựng đồ thị dòng điều khiển, xác định C, xây dựng test case

+ Xây dựng đồ thị luồng dữ liệu cho các biến

+ Xác định kịch bản kiểm thử cho đời sống các biến

```
int [] PGTB2(int a, int b, int c){  
1.     float x=0,x1, x2, detal;  
2.     if (a==0) return [0];  
3.     delta = b*b - 4 * a *c ;  
4.     if (detal <0)  
5.     {     x= [1];  
6.           return [x];  
7.     }  
8.     if (delta ==0)  
9.     {     x = (-b/(2*a));  
10.          return [x];  
11.     }  
12.     x1 = (-b + sqrt (detal)) / (2*a);  
13.     x2 = (-b - sqrt (detal)) / (2*a);  
14.     return [x1,x2];  
}
```